序列和集合的算法

序列与集合

- 序列须考虑元素的顺序，而集合则不必
- 集合中的元素不能重复出现，而序列则没有这样的限制
- 序列或集合中的元素取自全序集合（如整数集合、实数集合等），由此任意元素之间可以比较大小

二分搜索的几种形式

- 二分搜索
- 循环序列的二分搜索
- 二分搜索特殊下标
- 二分搜索长度未知的序列
- 重叠子序列问题
- 方程求解

二分搜索

- 问题：给定实数序列$x_1,x_2,\cdots,x_n$，满足$x_1 \leqslant x_2 \leqslant \cdots \leqslant x_n$。对某个实数z，试确定z是否在该序列中出现。若z出现，试确定下标i使$x_i=z$。
- 为简单起见，这里只查找一个满足$x_i=z$的下标i。
- 基本思想：通过一次查询检验中间数把搜索空间（大致）减半
- 算法复杂度：算法每进行一次比较，搜索的范围就减半，因此，在n个元素的序列中查找某个数值需要比较O(log n)次。

循环序列的二分搜索

- 如果在序列$x_1,x_2,\cdots,x_n$中，存在某个i使x_i是序列中的最小者，且序列$x_i,x_{i+1},\cdots,x_n,x_1,\cdots x_{i-1}$是递增的，则称序列$x_1,x_2,\cdots,x_n$是循环序列。
- 问题：对某个已知的循环排序的序列，找出其中最小元素的位置。为简单起见，假设该位置是唯一的。
- 思想：利用二分搜索，通过一次比较将序列长度减半。在序列中任取两个元素x_k和x_m，其中$k<m$。若$x_k<x_m$，i就不可能落在$k<i\leqslant m$中，因为x_i是序列中的最小数（注意，这里不能排除x_k）。另一方面，若$x_k>x_m$，i就一定落在$k<i\leqslant m$中，因为在这个区间内元素的顺序发生了跳变。这样一来，比较一次元素大小就能排除许多元素。通过选择恰当的k和m就能在O(log n)次比较后找到i。

```
Algorithm Cyclic_Binary_Search(X,n)
Input: X, a cyclically sorted array in the range 1 to n of
   distinct elements
Output: Position (the index of the minimal element)
begin
   Position:=Cyclic_Find(1,n);
end
function Cyclic_Find(Left,Right):integer;
begin
   if Left=Right then Cyclic_Find:=Left
   else
      Middle:=Left+ ⌊(Right-Left)/2⌋;
      //为什么不写⌊(Left+Right)/2⌋?
      if X[Middle]<X[Right] then
         Cyclic_Find:=Cyclic_Find(Left,Middle)
      else
          Cyclic_Find:=Cyclic_Find(Middle + 1,Right)
end
```

二分搜索特殊下标

- 问题：给定一个由不同整数$a_1,a_2,\cdots,a_n$按升序排列而成的序列，试确定是否存在某个下标i使得$a_i=i$。
- 思想：考察$a_{n/2}$值（假设n是偶数），如果它的值恰好是n/2，则问题得解。否则的话，若该值小于n/2，则由所有元素各不相同可知，$a_{n/2-1}$的值小于n/2-1，依次类推，前半段序列中的元素均不满足性质，可对后半段继续搜索。若该值大于n/2，则同样可以得到相似的结论。

```
Algorithm Special_Binary_Search(X,n)
Input: X, a sorted array in the range 1 to n of distinct
   integers
Output: Position (the index of satisfying
   A[position]=Position, or 0 if no such index exists)
begin
   Position:=Special_Find(1,n);
end
function Special_Find(Left,Right):
begin
   if Left=Right then
      if A[Left]=Left then Special_Find:=Left
                           else  Special_Find:=0
   else
      Middle:= Left+ ⌊(Right-Left)/2⌋;
      if X[Middle]<Middle then
         Special_ Find:= Special_ Find(Middle + 1,Right)
      else
         Special_ Find:= Special_ Find(Left,Middle)
end
```

二分搜索长度未知的序列

- 考虑一般的搜索问题，但序列长度未知。
- 此时不能把搜索空间减半，因为序列边界并不清楚。
- 查找大于等于z的某个x_i。如果找到了这个x_i，就可以在下标为1到i的范围内进行二分搜索了。
- 首先将z同x_1比较。若$z \leqslant x_1$，则z只可能等于x_1。借助归纳法，假设对某个$j \geqslant 1$有$z > x_j$，将z同x_{2j}比较，搜索空间加倍。若$z \leqslant x_{2j}$，可推得$x_j < z \leqslant x_{2j}$，然后再通过O(log j)次比较就能找到z。
- 总的来看，如果i是使$z \leqslant x_i$的最小下标，那么通过O(log i)次比较就能得到某个x_j，使得$z \leqslant x_j$，然后再通过O(log i)次比较最终得到i。

重叠子序列问题

- 令A和B是有限字母表上的字符序列，$A=a_1a_2\cdots a_n$，$B=b_1b_2\cdots b_m$，其中$m\leqslant n$。若存在下标$i_1<i_2<\cdots<i_m$，对所有的j，$1\leqslant j\leqslant m$，有$b_j= a_{i_j}$，则称B是A的子序列，即如果能把B中的字符按序嵌入A中，嵌入的位置不必连续，则B是A的子序列。
- 从头开始扫描A直到b_1第一次出现（如果存在的话），继续扫描直到b_2出现，以此类推。由于算法中包含一次对A和B的线性扫描，因而运行时间为O(m+n)。
- 对已知的序列B，定义B^i为B的每个字符连续出现i次所形成的序列。
- 问题：给定两个序列A、B，试确定i的最大值，使得B^i是A的子序列。

重叠子序列问题

- 对每个已知的i，构造序列B^i是轻而易举的。
- 对任意给定的i，也能够判定B^i是否为A的子序列。
- 如果B^j是A的子序列，那么对$1 \leqslant i \leqslant j$，$B^i$也是A的子序列。
- i的最大值不能超过n/m，否则序列B^i的长度就会超过A。
- 首先令$i=\lceil n/m \rceil/2$，检验B^i是否为A的子序列。然后根据结果，如果它是A的子序列，则i介于$\lceil n/m \rceil/2$和$\lceil n/m \rceil$间，否则就考虑1到$\lceil n/m \rceil/2$的情况。这样共需进行$\lceil \log(n/m) \rceil$次检验才能确定i的最大值，耗费的总时间为$O((n+m)\log(n/m))=O(n \log(n/m))$。

由重叠子序列问题得到的启示

- 每当要查找满足某个特定性质的最大i，首先需要一个能判断i是否满足该性质的算法。
- 如果i有上界：若i满足该性质，对1≤j≤i，j也满足，那么接下来就可以借助二分搜索来解决。
- 若i的上界未知：可以采取加倍法。即从i=1开始，对i的值加倍直至找到合适的范围。

方程求解

- 设欲解方程f(x)=0，其中f是可计算的连续函数。现已知x落在区间[a,b]中（a≤x≤b），且f(a)·f(b)<0。求该方程在指定精度下的解。
- 由函数的连续性可知区间[a,b]中一定有解。于是就可以利用二分搜索的变形二分法来求解。
- 取x_1=a+(b-a)/2，若f(x_1)=0（在给定的精度下），x_1就是方程的解。否则解一定在[a,b]的半区间[a, x_1]或[x_1,b]中，取区间端点函数值一正一负的那个区间继续进行上述过程，直到求得给定精度下的解。在k步之后，含解区间的长度为(b-a)/2^k。

- 推广：如何求sqrt(x)

二分搜索的几种形式

- 二分搜索
- 循环序列的二分搜索
- 二分搜索特殊下标
- 二分搜索长度未知的序列
- 重叠子序列问题
- 方程求解

内插搜索

- 二分搜索总是把搜索空间减半，从而保证其性能是对数级的。而当某一步所得到的值十分接近目标值时，在该值的邻域内搜索似乎比盲目地减半搜索空间更加合理。
- 内插搜索的性能不仅取决于序列的长度，还和序列自身的特点有关。对有些序列，内插搜索不得不一一检查其中的每个数，而对于数据分布相对均匀的序列，内插搜索就十分高效。
- 内插搜索的平均比较次数是O (log log n)。尽管内插搜索比二分搜索在性能上有指数阶的进步，但在实际性能上并无多大改善：1)除非n很大，否则logn已经足够小以至于其对数并不比其本身小多少；2)内插搜索需要更复杂的数值运算。

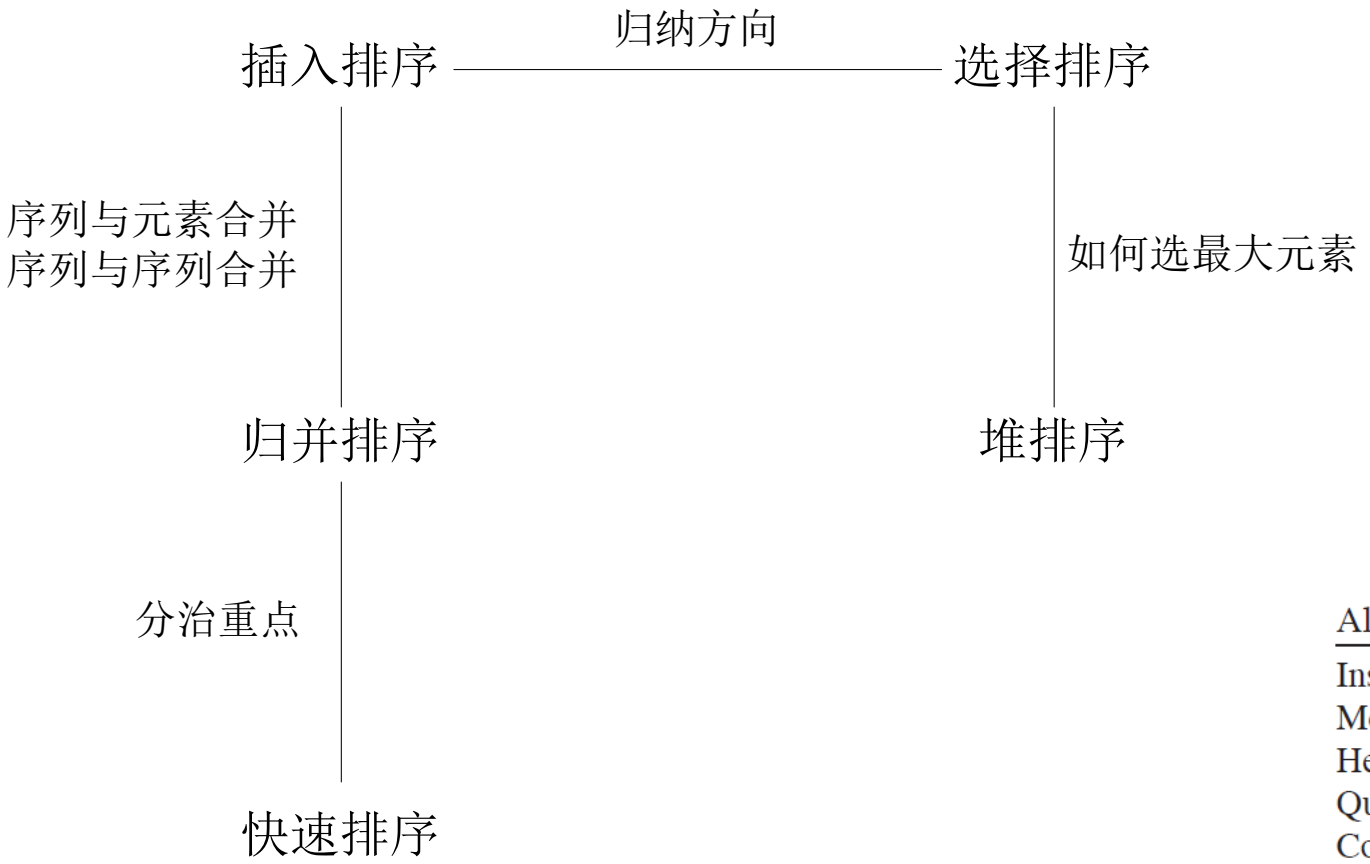
```
Algorithm Interpolation_Search(X,n,z)
Input: X (a sorted array in the range 1 to n), z (the search key)
Output: Position (an index i such that X[i]=z, or 0 if no such index
   exist)
begin
   if z<X[1] or Z>X[n] then position:=0
   else Position:=Int_Find(z,1,n)
end

function Int_Find(z,Left,Right):integer;
begin
   if X[Left]=z then Int_Find:=Left
   else if Left=Right or X[Left]=X[Right] then  Int_Find:=0
   else
      Next_Guess:=⌈Left+(z-X[Left])(Right-Left)/(X[Right]-X[Left])⌉;
      if z<X[Next_Guess] then
         Int_Find:=Int_Find(z,Left,Next_Guess-1)
      else
         Int_Find:=Int_Find(z,Next_Guess,Right)
end
```

排序

- 问题：对已知的n个数$x_1,x_2,\cdots,x_n$从小到大排序，也就是要找到由n个不同的下标所组成的序列$1\leqslant i_1<i_2<\cdots<i_m\leqslant n$，使得$x_{i_1}\leqslant x_{i_2}\leqslant\cdots\leqslant x_{i_m}$
- 为简单起见，假设数据均不相同。
- 如果某个排序算法除存储原始数据的数组之外不再借用额外的工作空间，则称之为原地置换排序算法。

排序



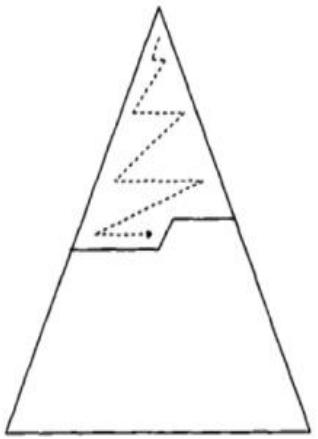
Algorithm	Worst-case running time	Average-case/expected running time
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Heapsort	$O(n \lg n)$	—
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$ (expected)
Counting sort	$\Theta(k + n)$	$\Theta(k + n)$
Radix sort	$\Theta(d(n + k))$	$\Theta(d(n + k))$
Bucket sort	$\Theta(n^2)$	$\Theta(n)$ (average-case)

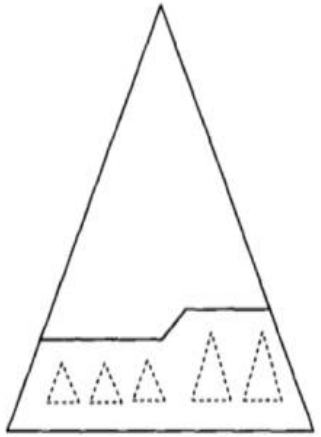
快速排序

- 快速排序算法在最坏情况下的运行时间为O(n^2)。
- 若始终选择中位数作为基准值，则时间复杂度为O(nlogn)。
- 快速排序算法对n个元素的数组进行排序时，平均比较次数为O(nlogn)。
- 当数组X中所有元素值相同时，运行时间为O(n^2)

堆排序

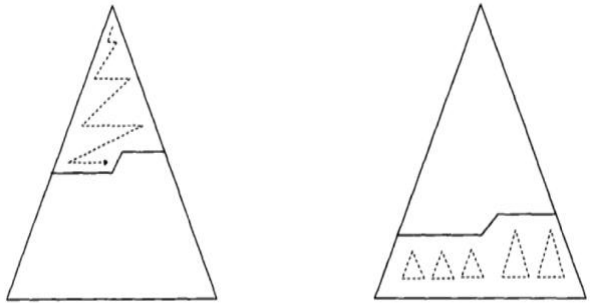
- 建堆：已知数组A[1..n]的元素按任意顺序排列，重新排列这些元素使其满足堆的性质。
- 自顶向下法和自底向上是两个很自然的建堆方法，它们分别对应着两种不同的对数组的扫描方式：自左向右或自右向左。





堆排序

- 从左到右对数组进行扫描（对应自顶向下建堆法）。
- 归纳假设(自顶向下)：数组A[1..i]是堆。
- 归纳基础是显见的，因为A[1]本身一定是堆。
- 如何把A[i+1]放到堆A[1..i]中去，即如何在堆中插入A[i+1]：将A[i+1]与其父节点比较，然后不断交换节点的位置，直到新的父节点大与其自身。最坏情况下的比较次数是$\lfloor \log(i+1) \rfloor$。



- 从右到左对数组进行扫描（对应自底向上建堆法）
- 试图说明数组A[i+1..n]是堆，然后如何在堆中插入A[i]。可是，A[i+1..n]并不代表某个堆，它对应着数个堆。
- 归纳假设(自底向上)：A[i+1..n]代表的所有树都满足堆的性质。
- A[n]本身是堆，归纳基础成立。数组A[$\lfloor n/2 \rfloor$+1..n]表示树上的叶节点，对应着A[$\lfloor n/2 \rfloor$+1..n]的树都是单节点树，自然满足堆的性质。因此只需从$\lfloor n/2 \rfloor$开始归纳。
- 考察A[i]，它最多只能有两个子节点(A[2i]和A[2i+1])，且都是其他某个堆的根节点。A[i]先同它最大的子节点比较，如果有必要就和那个最大的子节点交换。沿着树自上而下不断交换直到原先的A[i]值大于其现在两个子节点的值。由于A[i]的高度是$\lfloor \log(n/i) \rfloor$，最坏情况下的比较次数是$2\lfloor \log(n/i) \rfloor$。

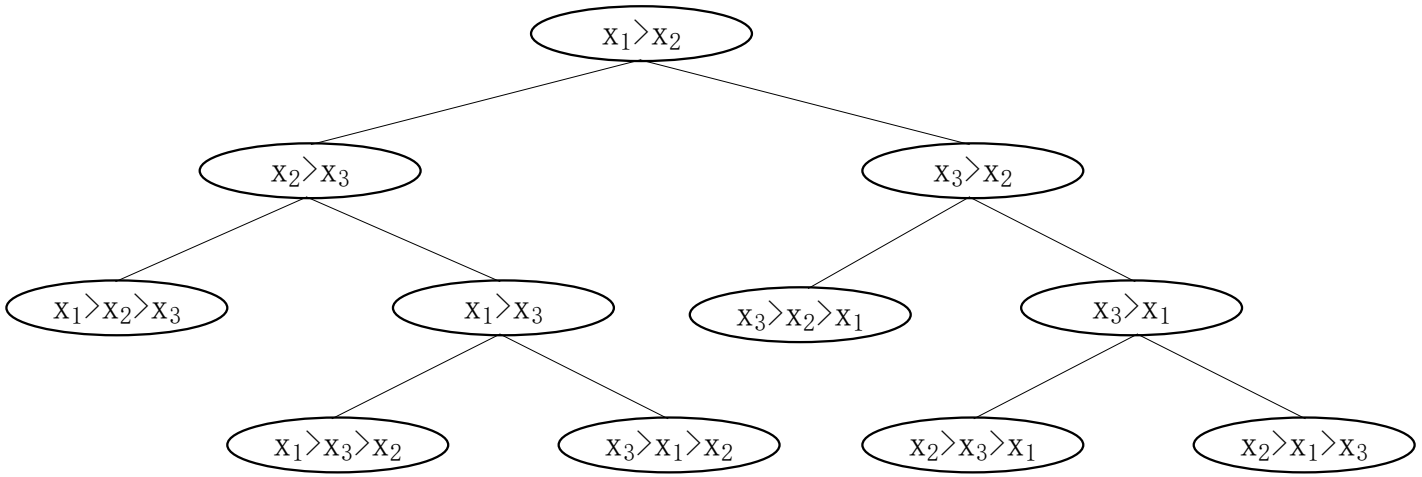
堆排序

- 自顶向下
- 第i步最多需要$\lfloor \log i \rfloor$次比较，故运行时间为O(nlogn)

- 自底向上
- 每一步比较次数最多是相应节点高度的两倍，所以算法复杂度最多是树上所有节点高度和的两倍。首先考察满二叉树，记H(i)是高度为i的满二叉树所有节点的高度之和，并把高为i的树看作两棵高为i-1的树再加上一个根节点。于是H(i)=2H(i-1)+i, H(0)=0。可以证明该递归关系式的解是$H(i)=2^{i+1}-(i+2)$。因为高为i的满二叉树有2^i-1个子节点，可推得满二叉树（即有2^k-1个节点的堆）自底向上建堆的算法复杂度是O(n)。那么为有n个节点的二叉树建堆，$2^k \leqslant n < 2^{k+1}-1$，其算法复杂度不会超过$2^{k+1}-1$个节点的堆，故仍为O(n)。
- 自底向上法比自顶向下法快主要是因为树底部的节点多于顶部，所以简化对底部节点的处理可得到更好的结果。

决策树

- 决策树是由内部节点和叶子节点构成的二叉树。每个内部节点都是一个查询，查询结果有两种可能，分别由该节点发出的两条边表示，每个叶子节点都是一种可能的输出。
- 设算法的输入是序列$x_1,x_2,\cdots,x_n$。计算过程从根节点开始，每个节点对输入进行查询，根据查询结果选择左、右分支，直到到达某个叶子节点，把该叶子节点的输出作为整个计算过程的输出。最坏情况下的运行时间是T的高度，相当于输入最多所需查询的次数。因此决策树事实上对应着算法，尽管它不能模拟所有的算法（比如说，决策树无法计算平方根），但是对于基于比较的算法而言，它的确是一个很合适的模型。由决策树所取得的下界意味着不存在比决策树更好、跟它的形式相同的算法。



决策树

- 定理6.1：任意一个排序算法的决策树高度为Ω(nlogn)。
- 此下界称为信息论下界，它和计算过程完全无关（请注意刚才甚至没有定义内部节点允许的查询种类），只和输出信息量有关。
- 问题下界的证明说明了基于比较的排序算法不可能比Ω(nlogn)快，不过可以通过利用关键字的特殊性质或者对关键字进行代数运算来加快排序速度。
- 如果问题的下界和某个算法的上界相等，那么这个下界就意味着算法不可能进一步优化。

计数排序

- 前提：假设有n个取值范围为1至k的元素，当k=O(n)时，运行时间O(n)
- 基本思想：对每一个x，求出小于x的元素个数。有了这个信息就可把x直接放到其在最终输出数组中的位置上

- 复杂度：O(n+k)
- 不出现输入元素间的比较，而利用输入元素的实际值来确定它们在数组中的位置
- 具有相同值的元素在输出数组中的相对次序与它们在输入数组中的相对次序相同－－稳定性

```
Algorithm CountingSort(A,n,k)
Input: A (an array in the range 1 to n)
Output: B (the array in the sorted order)
begin
  for i:=1 to k
      C[i]=0
  for j:=1 to n
      C[A[j]]=C[A[j]]+1  {C[i]=值为i的元素个数}
  for i:=2 to k
      C[i]=C[i]+C[i-1] {C[i]=值小于等于i的元素个数}
  for j:=n downto 1
      B[C[A[j]]]=A[j];
      C[A[j]]=C[A[j]]-1;
end
```

桶排序算法

- 假设有n个取值范围为1至m的元素。为它们分配m个桶，然后对每个i，根据x_i的值，把x_i放到相应的桶中去。最后按序扫描每个桶，并把桶中的元素收集起来。
- 算法复杂度为O(m+n)。一方面，若m=O(n)，算法就演化成线性时间排序算法；另一方面，若m相对于n很大，则O(m)也会很大，而且算法需要O(m)的存储空间，这比m自身很大更让人头疼。

桶排序算法的一个拓展

- 假设有n个元素，均匀分布在区间[0,1)上。
- 把区间[0,1)划分成n个相同大小的区间/桶，然后根据x_i的值，把x_i放到相应的桶中去。由于输入元素是均匀分布的，一般不会有很多数落在一个桶中。先对每个桶中的数进行排序，最后按序扫描每个桶，把桶中的元素收集起来。

```
for i:=1 to n
    将A[i]插入到表B⌊n*A[i]⌋中
for i:=0 to n-1
    用插入排序对表B[i]进行排序
将表B[0], B[1],…,B[n-1]按顺序合并
```

- 平均时间复杂性：O(n)
- 最坏时间复杂性：O(n^2)　数据不均匀，很多数落在一个桶里

桶排序算法->基数排序

- 假设元素都是k位数的较大整数，每一位的取值范围为0到d-1。
- 归纳假设：知道该怎样用<k位的元素进行排序。以字典序排序的序列，最高位决定了元素排列的顺序，同其他位无关。另一方面，如果两个元素的最高位相同，则由归纳假设，在最后一步之前，它们已经按正确的顺序排列。因此，仅需保证其排列顺序不变。
- 关键：同一个桶内的元素排列顺序不变。

- 基数排序是现在只有在计算机博物馆才能看到的卡片排序机所使用的算法。卡片有 80 列，机器可以在每一列的 12 个地方打孔。分拣机可以通过机械“编程”来检查一组卡片中每张卡片的某一列，并根据打孔的位置将卡片分配到 12 个仓位中的一个。然后，操作员就可以把卡片一张一张地收集起来，这样，第一处打孔的纸牌就会放在第二处打孔的纸牌上面，以此类推。
- https://www.columbia.edu/cu/computinghistory/sorter.html
- 应用：日期排序，先排日，然后月，最后年

基数排序

7467	6792	9134	9134	1239
1247	9134	1239	9187	1247
3275	3275	1247	1239	3275
6792	4675	7467	1247	4675
9187	7467	3275	3275	6792
9134	1247	4675	7467	7467
4675	9187	9187	4675	9134
1239	1239	6792	6792	9187

基数排序

```
Algorithm Straight_Radix(X,n,k)
Input: X (an array of  n integers, each with k digits, in the range 1 to d)
Output: X (the array in sorted order)
begin
   Assume all elements are initially in a global queue GQ;
   for i:=1 to d do      {d is the number of possible digits}
      initialize queue Q[i] to be empty;
   for i:=k downto 1 do
      while GQ is not empty do
         pop x from GQ;
         f:=the ith digit of x;
         insert x into Q[f];
      for t:=1 to d do
         insert Q[t] into GQ;
   for i:= 1 to n do
      pop X[i] from GQ
end
```

复杂度：O(nk)

秩

- 已知序列S= $x_1,x_2,\cdots,x_n$。若x_i是S中第k小的数，称x_i在S中的秩是k。通过排序，很容易就能知道每个数在序列中的秩，然而有些关于秩的问题不用排序也能解决。

最大和最小数

- 单独查找序列中的最大和最小数是很方便的。如果知道前n-1个数构成的序列中的最大数，就只需将这个最大数同第n个数作比较，从而得到这n个数中的最大数（查找1个数构成的序列中的最大数是显然的）。以上过程中从第2个数开始，每个数比较一次，因此共需比较n-1次。
- 问题：在已知的某个序列中查找最大和最小数。
- 直接法：分别查找最大数和最小数。这样一共要作2n-3次比较：n-1次找到最大数，n-2次找到最小数（因为此时可以排除最大数）。

最大和最小数

- 归纳法：设知道n-1个数时问题的解，现在求n个数时的解。那么就只需要把第n个数与当前找到的最大、最小数分别比较。这样，每个数需两次比较，可以推得总共仍需2n-3次。以不同顺序扫描序列中的数也不能改进其性能，这主要是由于序列中数的位置同问题是无关的。
- 扩展方案：已知n-2个数的解，对n个数求解。考察x_{n-1}和x_n，设MAX(min)是前n-2个数的最大（小）数，作三次比较便能得出新的最大最小数。首先比较x_{n-1}和x_n，然后将其中大数同MAX比较，小数同min比较。这样一来，总共只要大约3n/2次比较。
- 每次三个（或四个）仍要比较这么多次，用分治法解决这个问题最终也需要约3n/2次比较

查找第k小的数

- 问题：已知序列S= $x_1,x_2,\cdots,x_n$以及整数k，1≤k≤n，试查找S中第k小的数。
- 如果k接近1或者n，那么只要把查找最小数的算法运行k次便能找到第k小的数。这个方法大概需作kn次比较。排序算法一般都比这个原始的算法好，除非k是O(logn)或者n-O(logn)。
- 解题思想大致与快速排序相同。在快速排序中，序列被主元分成两个子序列，然后分别对这两个子序列递归排序。而现在可先确定第k小的数在哪个子序列中，然后只要对那个子序列递归查找就行了。不必考虑其余的数。

```
Algorithm Selection(X,n,k)
Input: X (an array in the range 1 to n), and k (an integer)
Output: S (the kth smallest element, the array X is changed)
begin
   if (k<1) or (k>n) then print “error”
   else S:=Select(1,n,k)
end

Procedure Select(Left,Right,k);
begin
   Partition(X,Left,Right);
   Let Middle be the output of Partition;
   if Middle-Left+1>k then Select(Left,Middle-1,k)
   else if Middle-Left+1=k then return X[Middle]
   else Select(Middle+1,Right,k-(Middle-Left+1))
end
```

复杂度：O(n)　　最坏呢？

加权中位数

- 加权中位数：n个数$x_1, x_2, \ldots, x_n$，相应正权值$w_1, w_2, \ldots, w_n$，$\Sigma w_i=1$，x_k为加权中位数满足以下条件

$$\sum_{x_i<x_k} w_i < \frac{1}{2} \qquad \sum_{x_i>x_k} w_i \le \frac{1}{2}$$

- 下面7个数，中位数是$x_5=4$，加权中位数是$x_7=6$，小于x_7的数有x_1, x_3, x_4, x_5, x_6共5个，权值之和为0.45，小于1/2，x_2大于x_7，权值为0.35也不大于1/2

i	1	2	3	4	5	6	7
x_i	3	8	2	5	4	1	6
w_i	0.12	0.35	0.025	0.08	0.15	0.075	0.2

- 当$w_i=1/n$时中位数与加权中位数一致

加权中位数

- 排序求加权中位数：按照x_i的值排序，然后从小到大计算权值的累计值直至加到x_k后大于等于1/2，则x_k为加权中位数，时间复杂性(nlogn)
- 类似求第k小元素：先求出中位数，计算小于中位数的元素的权值和以及大于中位数的元素的权值和，若符合加权中位数定义，则即为所求，若某一部分不符合定义，则在相应部分递归求解，但是需要修正权值

```
Weighted-Med(X)
    if n==1 return X[1]
    if n==2
        if w_1>=w_2 return x_1 else return x_2
    求出X的中位数x_k
    用x_k将数组X划分为两部分L={x_i<x_k}与R={x_i>x_k}，计算两个集合权值和为W_L和W_R
    若W_L<1/2且W_G<=1/2 return x_k  //加权中位数的定义
    若W_L>=1/2 则w_k=w_k+W_G，return Weighted-Med(L+x_k)
    w_k=w_k+W_L, return Weighted-Med(R+x_k)
```

- 时间复杂性：$T(n)=T(n/2+1)+\Theta(n)=\Theta(n)$

两个有序数组的中位数

- 两个有序数组A和B的长度都为n，寻找这2n个数的中位数
- 先合并排序，O(n)
- 比较A[n/2]和B[n/2]

1）A[n/2]=B[n/2]，则即为中位数

例1：A={1,2,3,4,5}，B={1’,2’,3’,4’,5’} //这些数字表示在各自数组中排在第几位，不代表值

若3=3’，则这10个数的大小关系为{1,2,1’,2’}<={3,3’}<={4,5,4’,5’}，3和3’都是中位数

例2：A={1,2,3,4}，B={1’,2’,3’,4’’}

若2=2’，则这8个数的大小关系为{1,1’}<={2,2’}<={3,4,3’,4’’}，2和2’都是中位数

2）A[n/2]>B[n/2]，则中位数是B[n/2..n]和A[1..n/2] 这n个元素的中位数，

例1：A={1,2,3,4,5}，B={1’,2’,3’,4’,5’}

若3>3’，则4至少排在第7位(比1,2,3,1’,2’,3’大），而2’最多排在第4位（比1,2,2’大），所以可以排除4,5,1’,2’，而{1,2,3}和{3’,4’,5’}的第3小数a必定比1’和2’大，从而排在原来10个数的第5位

例2：A={1,2,3,4}，B={1’,2’,3’,4’’}

若2>2’，则3至少排在第5位（比1,2,1’,2’大），2’最多排在第3位，所以可以排除3,4,1’,2’，而{1,2}和{3’,4’}的第2小数b必定比1’和2’大，所以排在原来8个数的第4位

3）A[n/2]<B[n/2]，则中位数是A[n/2..n]和B[1..n/2]这n个元素的中位数

从而T(n)=T(n/2)+1=O(logn)

另一个思路

- 假设中位数属于集合A，A[k]=m为2n个数的中位数
- 这意味着A中有k个元素小于等于m，有n-k个元素大于等于m
- A和B中有n个元素小于等于m，n个元素大于等于m
- 从而B中有n-k个元素小于等于m，n-(n-k)=k个元素大于等于m
- 所以可以检查是否存在B[n-k]<=A[k]<=B[n-k+1]
- 注意边界情况k=n，此时只需检查是否A[n]<=B[1]
- 如果中位数在A中，但是不是A[k]，如果中位数是A[k’]，且k’<k，那么A[k]大于中位数，并且B[n-k+1]<A[k]。反之，如果中位数是A[k”]，且k”>k，则A[k]小于中位数，并且A[k]<B[n-k]
- 因此可以通过二分搜索来查找是否存在A[k]使得1）k<n且B[n-k]<=A[k]<=B[n-k+1]或者2）k=n且A[k]<=B[1]，如果存在这样的A[k]，即为2n个数的中位数，否则中位数在B中，可做类似分析
- 时间复杂性为O(logn)

```
Two-Array-Med(X,Y,n)
    median=Find-Med(X,Y,n,1,n)
    if median==NotFound
        median=Find-Med(Y,X,n,1,n)
    return median

Find-Med(A,B,n,low,high)
    if low>high return Not-Found
    k=low+⌊(high-low)/2⌋
    if k==n and A[n]<=B[1] return A[n]
    if k<n and B[n-k]<=k<=B[n-k+1] return A[k]
    if A[k]>B[n-k+1] return Find-Med(A,B,n,low,k-1)
    return Find-Med(A,B,n,k+1,high)
```

数据压缩

- 问题：已知某个文本文件（字符串序列），为其设计一种编码方式使之满足前缀限制且使编码后的文本长度最短。
- 记字符为$C_1,C_2,\cdots,C_n$，相应字频(每个字符在文本中出现的次数，通过对典型文本进行统计而得)为$f_1,f_2,\cdots,f_n$。若某种编码方式E中，用长度为s_i的位串S_i表示C_i的话，文件F用编码方式E编码后的长度为$L(E,F)=\sum s_i f_i$
- 目标：设计一种满足前缀限制的编码方式E，且L(E,F)最小。
- 解码时按序扫描序列的每一位直到得到对应于某个字符编码的位串。为此考察二叉树，每个非叶节点有一条标着0、另一条标着1的两条发散边，每个叶子节点对应着一个字符，从根节点到叶子节点的路径就是这个字符的编码。当扫描文件的过程中到达叶子节点时，就能地确定相应的字符。
- 问题：如何构造这棵树，使L(E, F)最小。

数据压缩

- 令C_i和C_j是字频最小的两个字符，则存在一棵使L(E，F)最小的树，使C_i和C_j对应着离根节点最远的叶子节点。否则，若存在某个字频更高的字符在树上的位置低于这些字符，那么它就可以同C_i或C_j交换从而减小L(E，F)。因为树上每个节点不是有两个子节点就是无子节点，于是可以断定C_i和C_j是两个并列的节点。
- 用一个新的字符代替C_i和C_j，这个新字符记作C_{ij}，其字频为f_i+f_j。
- 现在问题有n-1个字符(n-2个原来的和一个新的)，这样就可以用归纳假设解决了。

```
算法 Huffman_Encoding(S, f)
输入：S (字符串)和f（字频数组）
输出：T（S的霍夫曼树）
根据字频将每个字符插入堆H;
while H不空 do
   if H只包含一个字符X  then
      把X作为T的根节点
   else
      取字频最低的两个字符X和Y，从H中将它们删去;
      用新字符Z取代X和Y，且新字符Z的字频是X和Y的字频之和;
      把Z插入H;
      把X和Y作为T中Z的子节点{Z还没有父节点}
```

算法复杂度：构造树的过程中，每个节点都用常数时间。插入和删除操作分别用了O(logn)步。因此，算法总的运行时间是O(nlogn)。

序列比较

- 问题：把一个序列$A=a_1a_2\cdots a_n$变成另一个序列$B=b_1b_2\cdots b_m$的最少修改步数
- 修改方法：1) 插入——把某个字符插入字符串，2) 删除——从字符串中删除某个字符，3) 替换——把某个字符替换成另一个字符。
- 记前缀子串$a_1a_2\cdots a_i(b_1b_2\cdots b_i)$为A(i)(B(i))，问题就转化为用最少的修改步数将A(n)改为B(m)。

序列比较

记C(i, j)为A(i)变成B(j)的最小代价，建立C(n, m)和某个C(i, j)之间的关系，这里i、j小于n、m。

删除：如果A到B的最小改动需要删除a_n，那么最佳方法就是先把A(n-1)变成B(m)，然后再删除这个字符。换句话说，C(n, m) = C(n-1, m)+1。

插入：如果A到B的最小改动需要插入某个字符和b_m匹配，那么就有C(n, m) = C(n, m-1)+1。也就是（通过归纳）先把A(n)以最小代价变成B(m-1)，然后再插入一个跟b_m相同的字符。

替换：如果用a_n替换b_m，那么首先要找到从A(n-1)到B(m-1)的最小改动，接下来若$a_n \neq b_m$，还要加1。

匹配：如果a_n和b_m相同，则C(n, m) = C(n-1, m-1)。

$$c(i,j)=\begin{cases}0 & a_i=b_j\\ 1 & a_i\neq b_j\end{cases}$$

$$C(n,m)=\min\begin{cases}C(n-1,m)+1 & \text{delete } a_n\\ C(n,m-1)+1 & \text{insert for } b_m\\ C(n-1,m-1)+c(n,m) & \text{replacing V matching } a_n\end{cases}$$

$$C(i,0)=i$$

$$C(0,j)=j$$

把一个规模为(n, m)的问题归纳到三个规模仅比原问题略小的子问题。

总的子问题数并不多。不会超过nm。

动态规划!

序列比较

```
Algorithm Minimum_Edit_Distance(A,n,B,m)
Input: A (a string of size n), and B (a string of size m)
Output: C (the cost matrix)
Begin
    for i:=0to n do C[i,0]:=i;
    for j:=1 to m do C[0,j]:=j;
    for i:=1 to n do
        for j:=1 to m do
            x:=C[i-1,j]+1;
            y:=C[i,j-1]+1;
            if ai=bj then z:=C[i-1,j-1]
            else  z:=C[i-1,j-1]+1;
            C[i,j]:=min(x,y,z);
end
```

复杂度为O(nm)

$$C(n,m)=\min\begin{cases} C(n-1,m)+1 & \text{delete } a_n \\ C(n,m-1)+1 & \text{insert for } b_m \\ C(n-1,m-1)+c(n,m) & \text{replacing V matching } a_n \end{cases}$$

$$C(i,0)=i$$

$$C(0,j)=j$$

查找众数

- 令E是整数序列$x_1,x_2,\cdots,x_n$。E中x的重数是x在E中出现的次数。如果某个数z的重数大于n/2，则它就是E中的众数（majority）。
- 问题：已知一数列，判断是否存在众数。若存在，则求出众数。
- 蛮力搜索
- 排序算法：排序后进行统计。
- 中数查找法：如果众数存在，它肯定等于中数（中数是第(n/2)小的数，而众数的出现次数超过n/2）。所以一旦找到中数，就能统计它出现的次数。

查找众数

- 删除两个不同的数后，原来序列中的众数仍为新序列中的众数。（否命题不成立：1、2、5、5、3中没有众数，但如果删除1和2，5就成了新的众数。）
- 按出现的顺序依次扫描。引入变量C（候选者）和M（重数）。当考察x_i时，C是$x_1,x_2,\cdots,x_{i-1}$中众数的候选者，M等于C当前已出现的次数减去被删除的次数，即$x_1,x_2,\cdots,x_{i-1}$被分成大小分别为2k和M的两组，其中2k+M=i–1。第一组有k对不同的数，第二组包含已至少出现M次的C。如果在$x_1,x_2,\cdots,x_{i-1}$中存在众数，在不断删除后，它必定能胜出并等于C。（C不是众数，也有可能在删除后胜出。）考察x_i，将它同C比较，根据比较结果增加或减少C出现次数。若不存在候选的众数（M=0），则C=x_i，M=1。最后只剩一个候选者C，统计其出现次数便能判定它是否为众数。

```
Algorithm Majority(X,n)
Input: X (an array of size n of positive numbers)
Output: Majority (the majority in X if it exists, or -1 otherwise)
begin
   C:=X[1];    M:=1;
   for i:=2 to n do
      if M=0 then
         C:=X[i];     M:=1;
      else
         if C=X[i] then M:=M+1
         else M:=M-1;
   if M=0 then Majority:=-1
   else
      Count:=0;
      for i:=1 to n do
         if X[i]=C then Count:=Count+1;
      if Count>n/2 then Majority:=C else Majority:=-1;
end
```

算法复杂度：找到候选者需n-1次比较，最坏情况下判定该候选者是否为众数也需要n-1次比较。因此，总共需2n – 2次比较。

频繁元素

- 频繁元素是指输入数据中有些元素会出现多次，希望找到出现最频繁的t个
- 如果输入数据不是很多，可以对每一个不同的元素设立一个计数器，数据输入时，相应的计数器加1。这样的算法是一个精确的算法，但需要的内存量可能会比较大，尤其是数据种类很多时，并且输入数据的数量也很多时，这一矛盾就会更加突出。

频繁元素-- Misra-Gries算法

- Misra-Gries算法是一种计算频繁元素的近似算法，该算法需要事先规定计数器的最大数量k，即只在内存中保留k个计数器。当有数据输入时，检查是否已经为其分配了计数器，这时共有三种不同的情况需要处理：
- 1）有对应的计数器，则相应的计数器加1
- 2）没有相应的计数器，并且已有的计数器数量小于k，则为该数据分配一个新的计数器，其值设为1。
- 3）没有相应的计数器，并且已经有了k个计数器，则将所有的计数器数值减1，并删除值为0的计数器，新数据也被丢弃。

输入：1，5，3，1，2，1，2，3，7，1
k=3，则计算过程为：

输入	计数器<数据，次数>
1	<1,1>
5	<1,1> <5,1>
3	<1,1> <5,1> <3,1>
1	<1,2> <5,1> <3,1>
2	<1,1>
1	<1,2>
2	<1,2> <2,1>
3	<1,2> <2,1> <3,1>
7	<1,1>
1	<1,2>

性能分析

- Misra-Gries算法是一种可以用于寻找频率大于n/(k+1)的元素的空间亚线性的近似算法。
- 如需获得准确的结果，则需要把这些算法结束时还留在内存中的计数器置0，再次扫描所有数据，并对相应计数器做计数工作。第二轮计数完毕后，检查各个计数器的值是否超过n/(k+1)。但两次扫描也意味着需要存储所有的数据，不再是一个空间亚线性的算法了。
- 频繁问题的一个特例是找出出现次数超过一半的元素，即寻找众数，只需将Misra-Gries算法中的k设为1即可

最长递增序列

- 给定一个由不同整数组成的序列，求其最长递增序列LIS
- 归纳假设1：给定某个长度小于m的序列，知道如何求它的某个最长的递增序列
- 归纳假设2：给定某个长度小于m的序列，知道如何求它的所有最长的递增序列
- 归纳假设3：给定某个长度小于m的序列，知道如何求它的某个最长的递增序列，使得其它最长递增序列的末尾的数都不比这个序列末尾的数小
- 归纳假设4：给定某个长度小于m的序列，知道如何对任意k<m-1求出BIS(k)，如果存在的话

最长递增序列

给定x_m，设前m-1个数字的LIS长度为s

for i=s downto 1

  if BIS(i).last<x_m break

end for

if i=0 then BIS(1)=x_m

if i=s then BIS(s+1)=BIS(s)+x_m

if 0<i<s then BIS(i+1)=BIS(i)+x_m (即BIS(i).last<x_m<BIS(i+1).last)

其中i的查找可用二分查找

所以每个x_m需要O(logm)次比较

总的运行时间为O(nlogn)

查找集合中最大的两个元素

- 给定有n个元素的集合S，求其中最大的和第二大的元素
- 直接法：2n-3
- 常规的分治法：3n/2-2
- 多花一些分的时间的分治法：n+logn-2
- 第二大元素的候选者集合：n+logn-2

计算多重集合的模

- 求给定的多重集合(元素不必互异)S的模
- 排序
- 直接归纳
- 分治法(利用中数)：O(nlogn)
- 分治法(改变归纳基础)：O(nlog(n/M))